Ship Compute or Data? Why Not Both?

Jie You, Jingfeng Wu, Xin Jin, Mosharaf Chowdhury

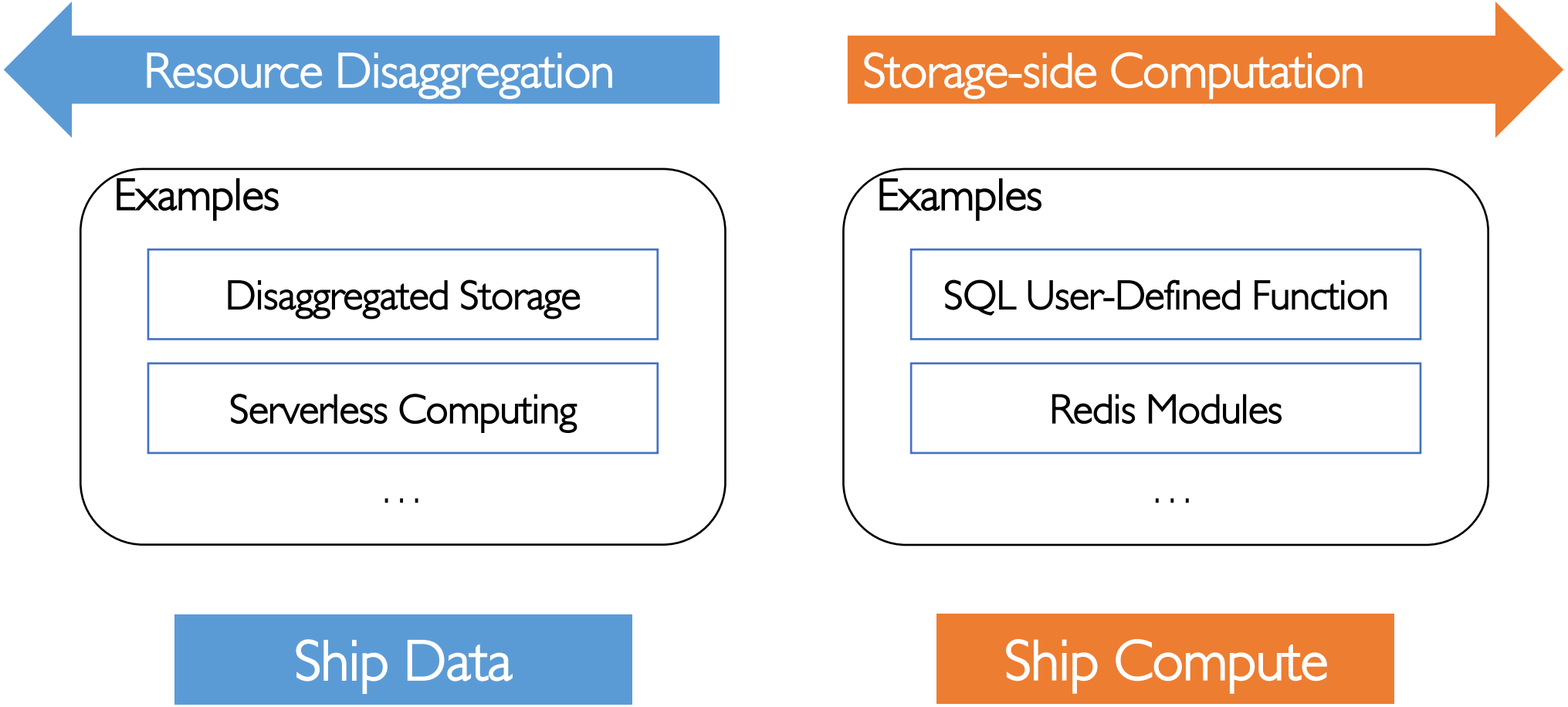
SymbioticLab

University of Michigan

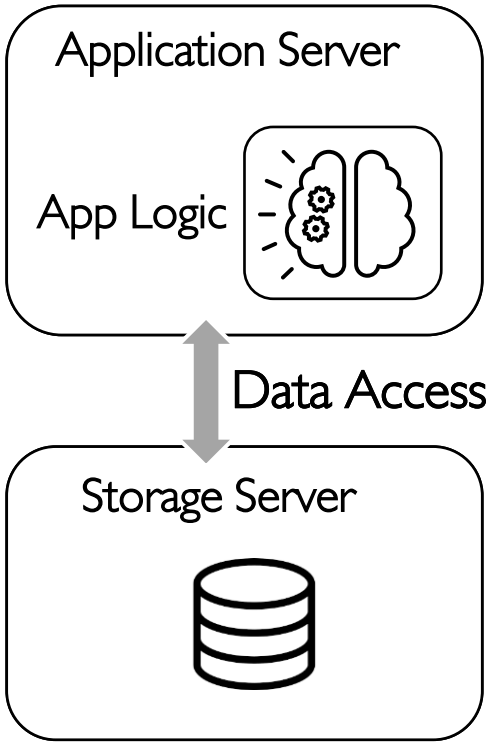
Johns Hopkins University

北京大学 Peking University

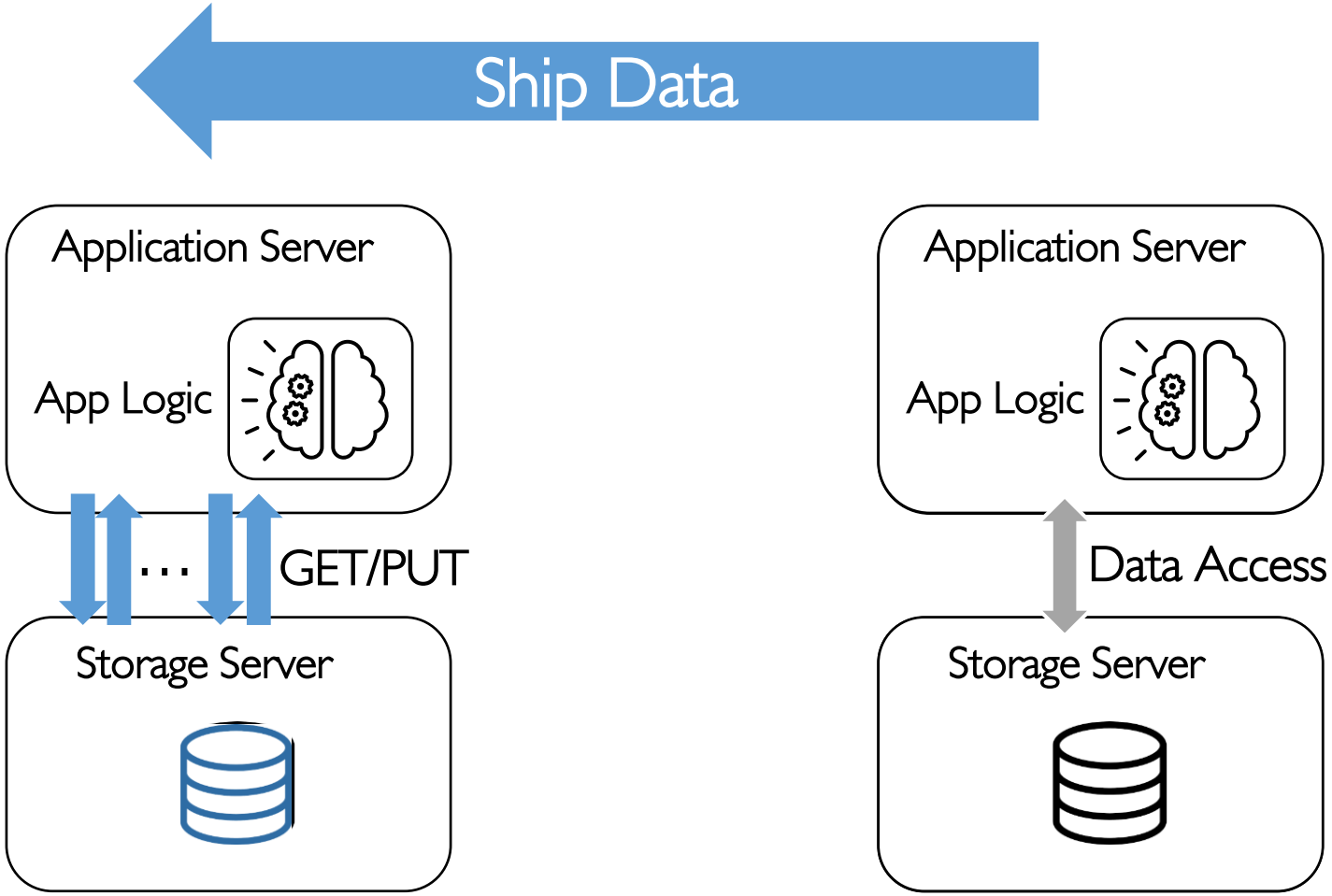
Two Trends of System Design



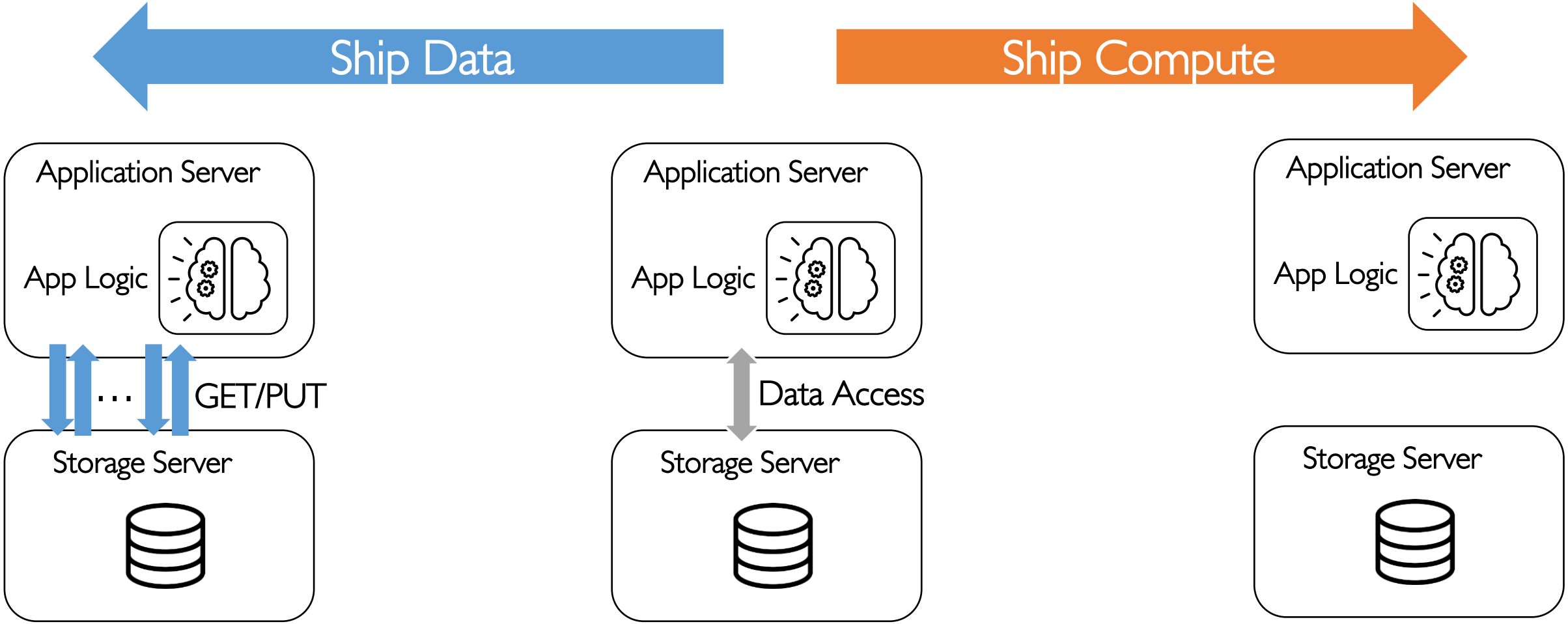
Ship Data OR Ship Compute



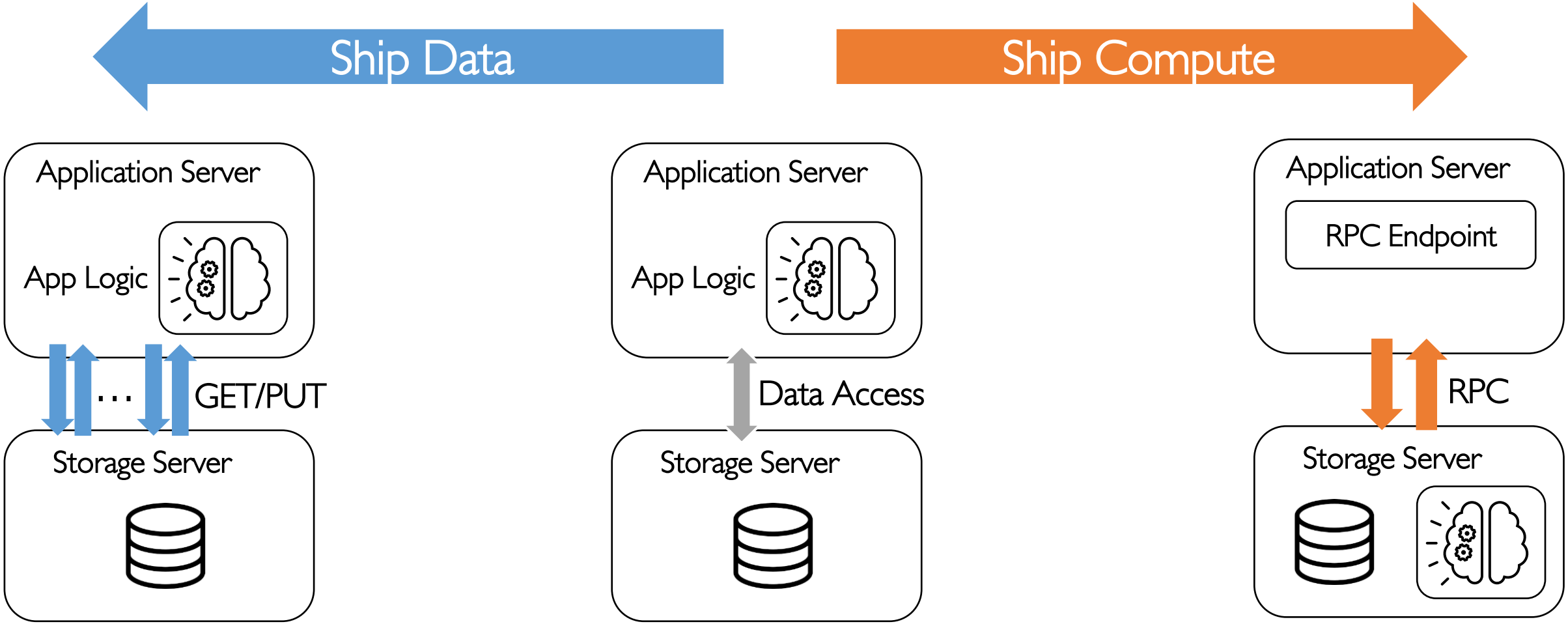
Ship Data OR Ship Compute



Ship Data OR Ship Compute



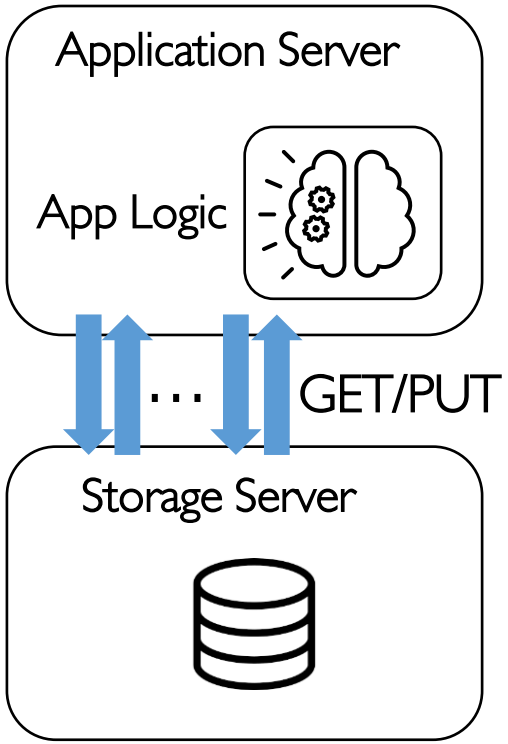
Ship Data OR Ship Compute



Ship Data OR Ship Compute

Ship Data

Ship Compute



Pros:

- Flexible provisioning
- Elastic Scaling

Cons:

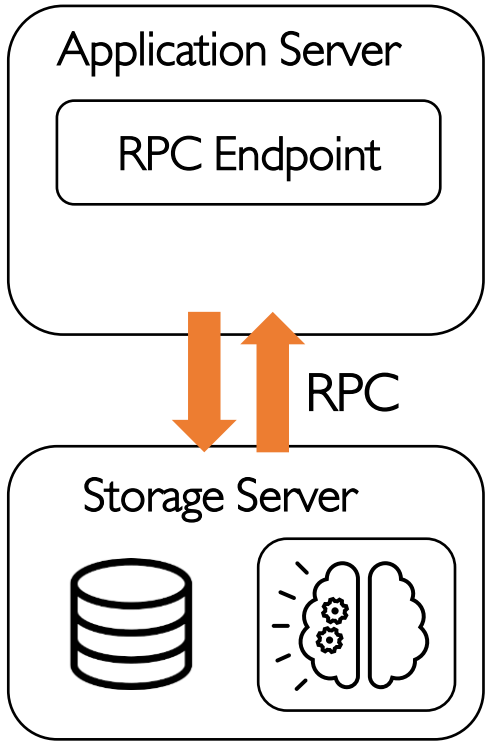
- More network traffic
- Multiple network round-trips for a single request (latency inflation)

Pros:

- Less network round-trips
- No marshalling overhead

Cons:

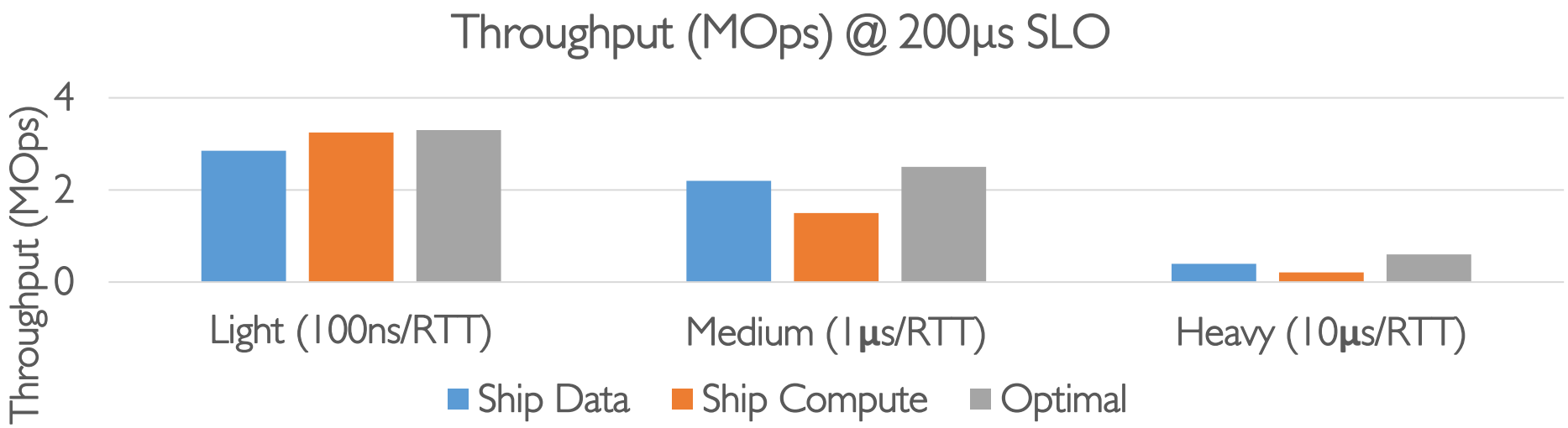
- Storage server can get overloaded



No One-Size-Fits-All Solution

Ship Data AND Ship Compute

- Measurement:
  - Workload: synthetic w/ configurable computation time and round-trips
    - Optimal: Best of sweeps across combinations of (ShipData%, ShipCompute%)



- Intuition: combining both → more CPUs utilized → higher throughput

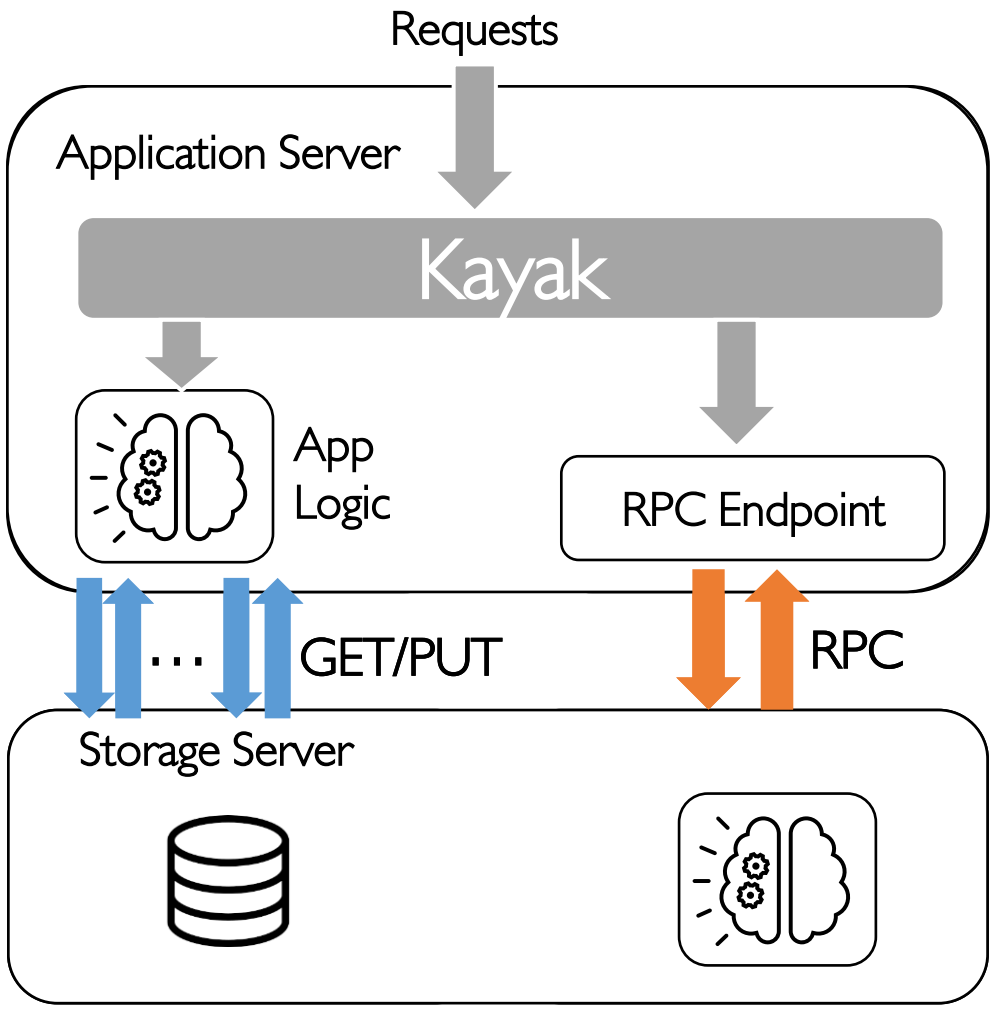
Kayak

*Ship Data **AND** Ship Compute*

1. Maximizes throughput under SLO constraints	Latency-driven rate limitation and request arbitration
2. Adaptively arbitrates between shipping compute and shipping data w/o workload knowledge	Dual-loop control with proved convergence

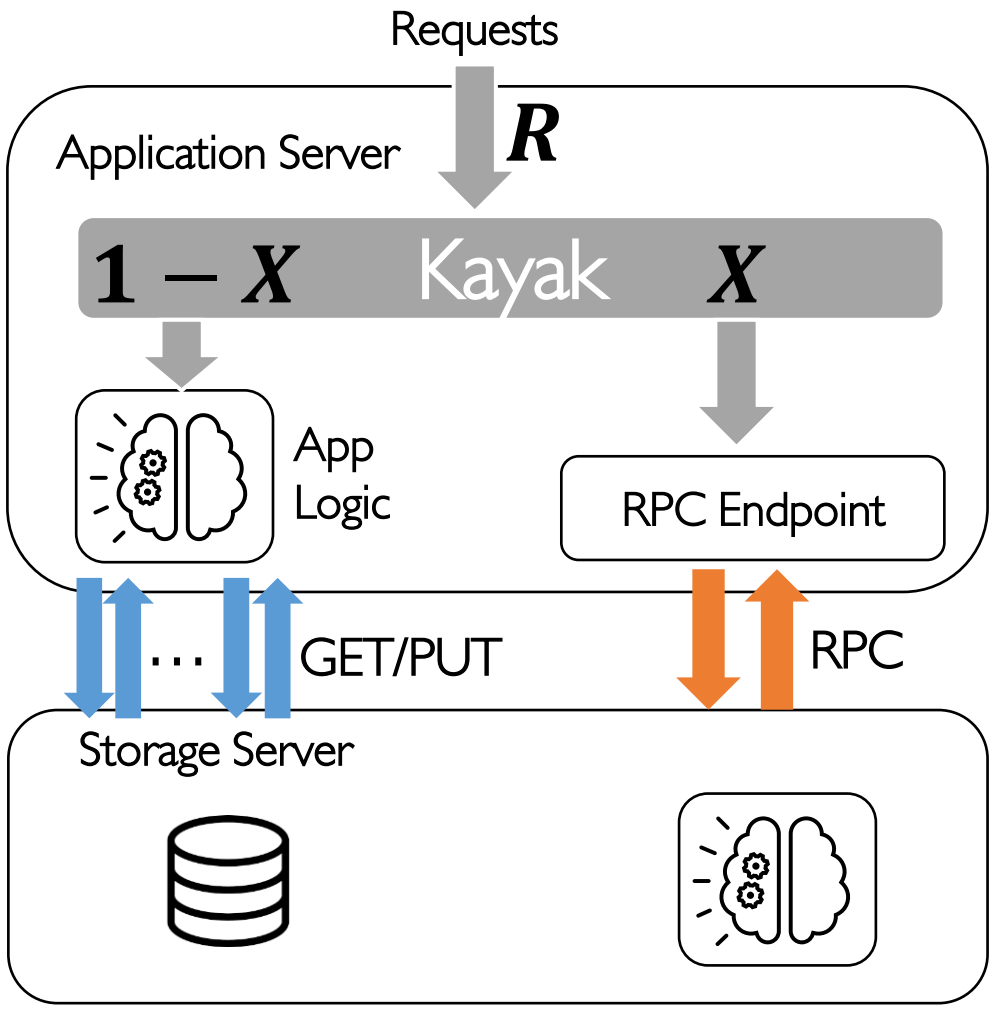
Kayak Overview

- **Combines** shipping data and shipping compute
- **Arbitrates** between shipping data and shipping compute
- **Applies** adaptive rate limitation to maximize throughput under SLO constraints



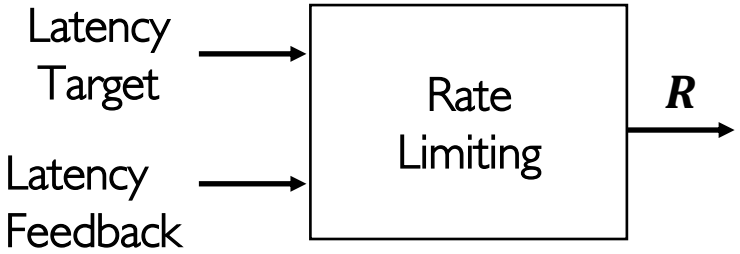
Algorithm Overview

- Kayak proactively tunes:
  - $\boldsymbol{R}$: The request rate limitation
  - $\boldsymbol{X}$: The fraction of requests to be executed using RPC/KV
- Challenges:
  - No closed-form expression of $(\boldsymbol{X}, \boldsymbol{R}) \rightarrow Latency$
  - Tuning both $\boldsymbol{X}$ and $\boldsymbol{R}$ simultaneously is hard



Algorithm Design

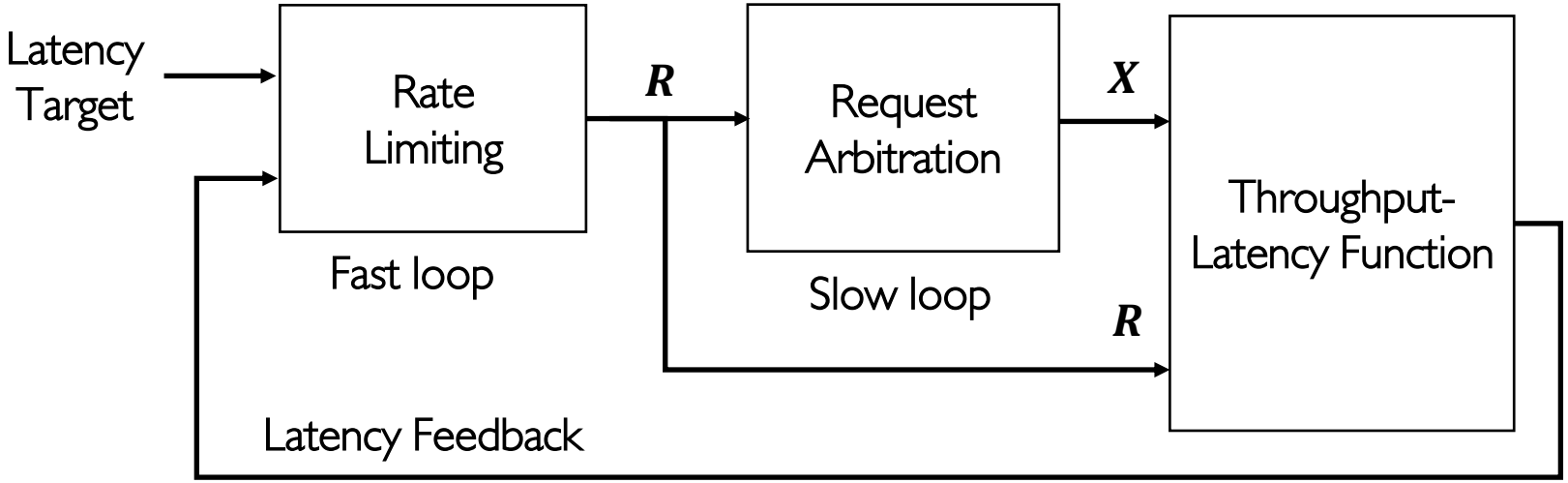
- Solution:
- 1. Optimize $\boldsymbol{X}$ and $\boldsymbol{R}$ separately via control loops
  - $\boldsymbol{R}$: Rate limiting control loop
  - $\boldsymbol{X}$: Request arbitration control loop
- 2. Combine two control loops together



$\boldsymbol{R}$: The request rate limitation
$\boldsymbol{X}$: The fraction of request to be executed using RPC/KV

Dual Loop Control

- Our Solution: R-X dual loop control
  - Rate limiting loop before request arbitration loop
  - The other way around is unstable
- Proved convergence of $\boldsymbol{R}$ and $\boldsymbol{X}$

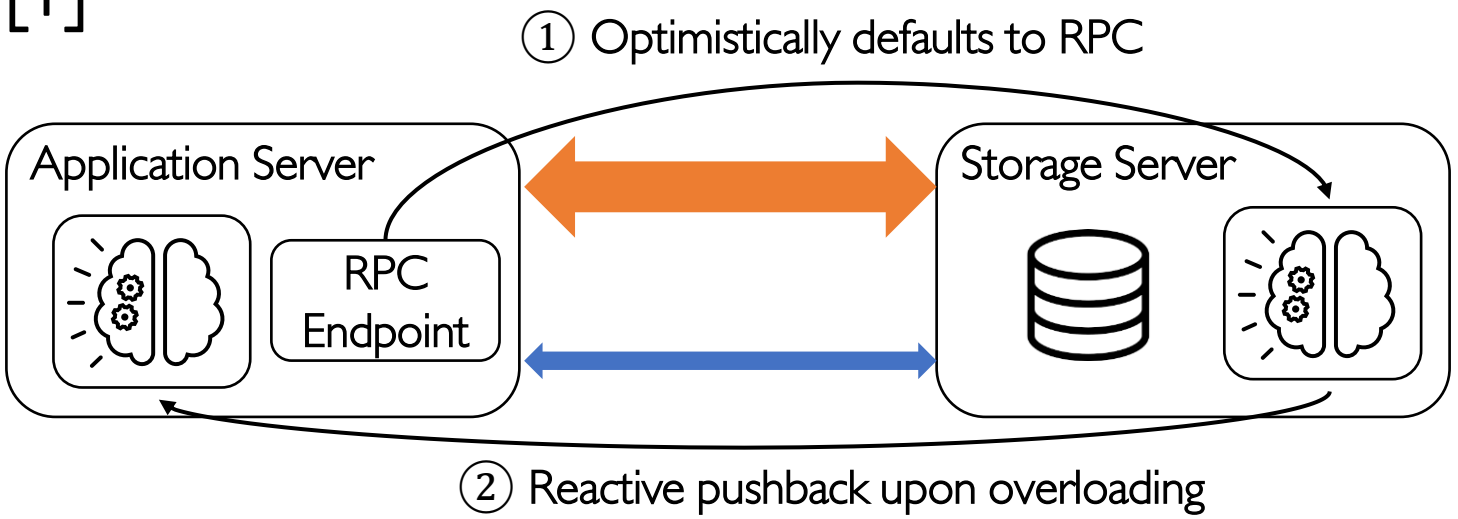


Evaluation

Workloads

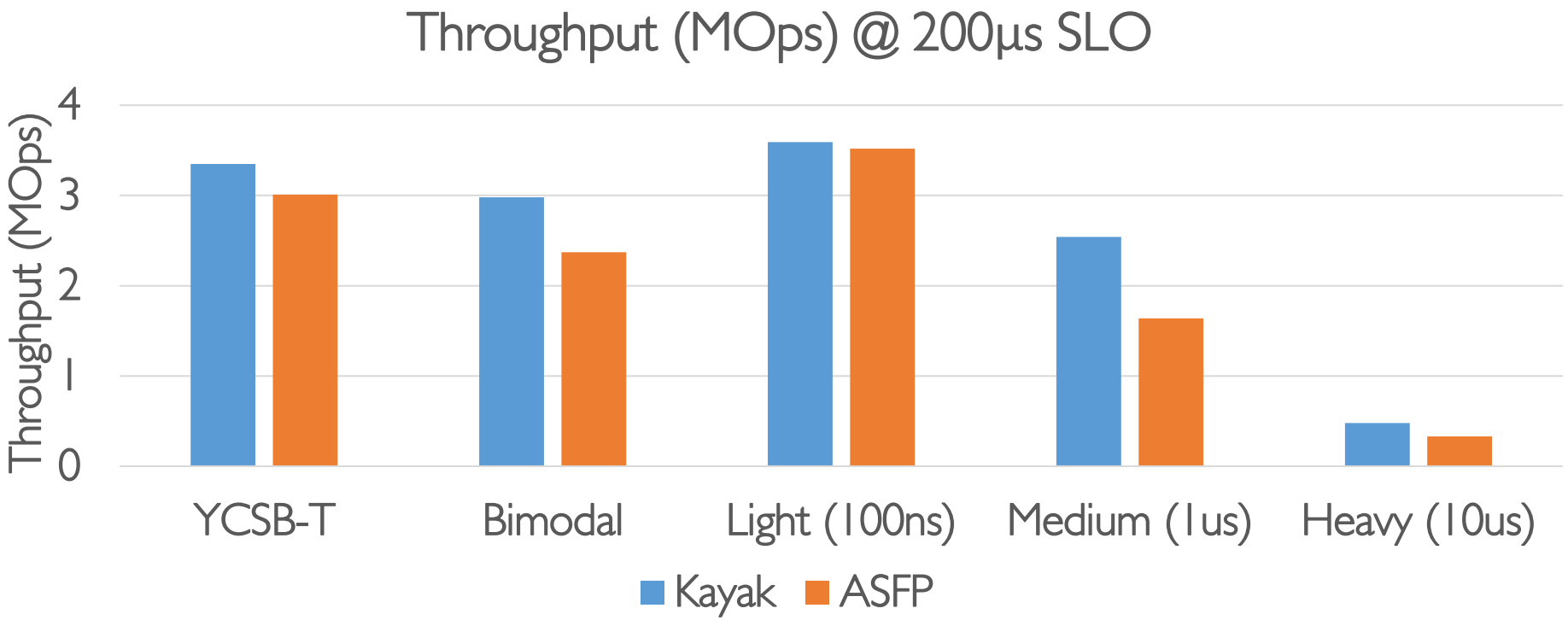
Workload	Comp. Time (µs/RTT)	Transactional	Time Varying
Synthetic	0.1~10		
Bimodal	0.1~1		✓
YCSB-T	<0.1	✓	

Baseline: ASFP [1]



[1] Adaptive Placement for In-memory Storage Functions (ATC '20)

Performance



12.2%-63.4% improvement

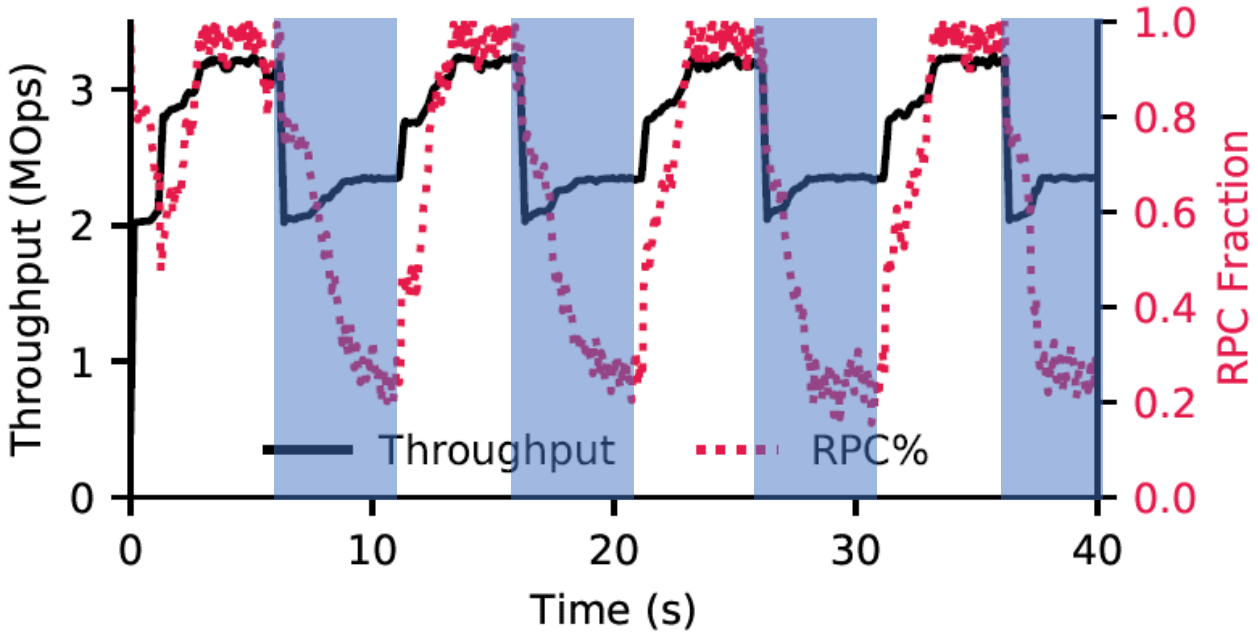
Higher improvements w/ compute-intensive workloads

Convergence of Dual Loop Control

- Workload:
  - Bimodal: Switching between (1us, 100ns), w/ an interval of 5s

Medium(1us/RTT)

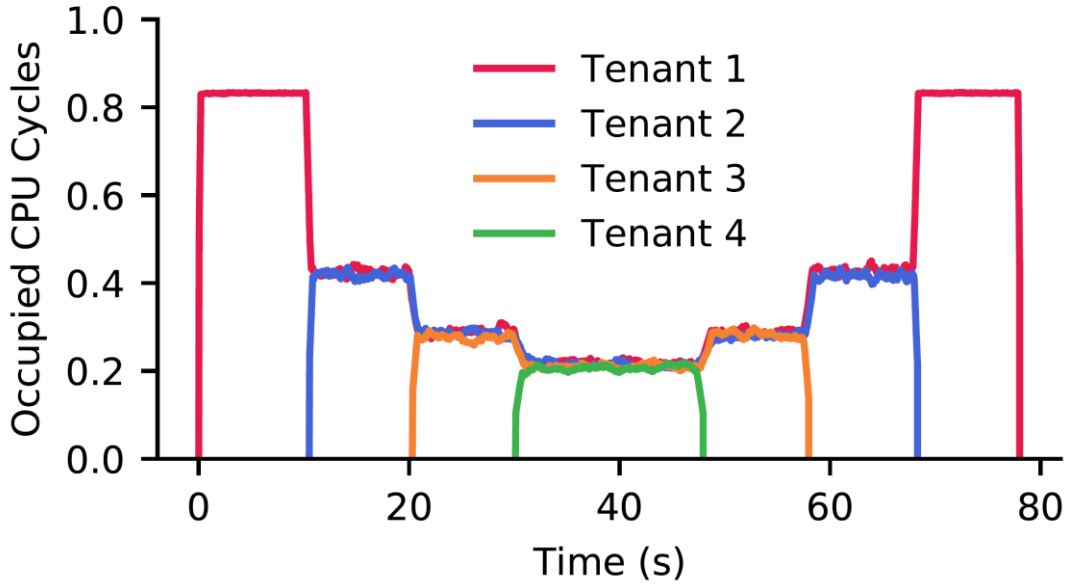
Light (100ns/RTT)



Quick and robust convergence

Fairness across Multiple Tenants

- Setup:
  - 4 tenants running on 4 separate applications servers
  - Single shared storage server



Fair sharing of Storage Server CPU

Ship Compute **AND** Ship Data

- Kayak
  - Proactively arbitrates between shipping data and shipping compute w/o workload knowledge
  - Maximizes throughput under latency SLO constraint
  - Ensures fairness across multiple tenants

- Thank you

https://github.com/SymbioticLab/Kayak